

ABOUT SPRINTS

1.1 Strategia generale

1. Ogni Sprint del processo di produzione deve definire in modo univoco ed esplicito il proprio obiettivo (**goal**). Non sono accettabili sprint privi di un focus chiaro e dichiarato.
2. Ogni Sprint successivo allo Sprint0 deve:
 - **analizzare i requisiti** relativi al **sottoinsieme** di funzionalità da sviluppare in quello Sprint, mediante una opportuna **interazione con il committente**,
 - **analizzare le problematiche** che sorgono dai requisiti, e individuare le **strategie/tecnologie** da adottare ritenute più opportune, tenendo conto delle conoscenze del team e dei tempi e dei costi correlati allo sviluppo;
 - definire, in modo comprensibile alla macchina, un insieme di **TestPlans** come premessa allo sviluppo del codice
 - **produrre un deliverable** (un prodotto software) comprensibile alla macchina ed eseguibile, che implementi le funzionalità previste dallo Sprint e capace di superare i TestPlans definiti in precedenza.

1.2 Costruire vs. Esprimere

Per rendere qualcosa comprensibile a un computer, occorre utilizzare un linguaggio di programmazione.

Un linguaggio di programmazione *general purpose* permette di esprimere la **costruzione** di un qualsiasi sistema software (computabile).

- **Costruire**: È l'atto tecnico dell'implementazione pura di una funzionalità o di un componente.
- **Esprimere**: È un concetto semantico di livello superiore. Significa dichiarare la natura intrinseca e i vincoli architetturali di un componente in modo nativo affinché la macchina li comprenda immediatamente e possa 'costruire' il componente senza bisogno di ulteriori interpretazioni.

i la necessità di esprimere

I linguaggi di programmazione ad oggetti non presentano costrutti utili ad esprimere direttamente concetti tipici in un sistema software distribuito, come la concorrenza, la distribuzione e la comunicazione/interazione tra componenti software, che risultano essere più assimilabili a processi, piuttosto che non a oggetti (POJO).

Questa difficoltà è alla base della **nascita di nuovi linguaggi di programmazione** ad alto livello per sistemi distribuiti, come *Erlang*, *Go*, *Rust* e *Scala* (si veda anche Oltre gli oggetti).

1.3 Componenti software

In sintesi, ricordiamo che oggi molti linguaggi di programmazione general purpose possono esprimere componenti software nella forma di:

- funzioni
- oggetti
- processi

Nei linguaggi più diffusi non esiste però la parola chiave nativa (il costrutto linguistico) **service**. Di conseguenza, i servizi si possono *costruire* (tramite design pattern, configurazioni di rete, socket o framework), ma non si possono esprimere in modo diretto.

1.4 Oggetti vs. servizi

- Un **oggetto** (POJO) è un'entità software che incapsula dati e comportamenti; non è però un'entità autonoma, ma dipende da un ambiente di esecuzione (JVM, CLR, ecc.) per essere eseguito.

(Ri)usare un oggetto scritto in Java è facile da parte di un altro oggetto scritto in Java, ma è difficile da parte di un oggetto scritto in C# o Python.

- Un **servizio** è un'entità software autonoma, che può essere eseguita in modo indipendente da un ambiente di esecuzione specifico. Un servizio può essere **scritto in qualsiasi linguaggio di programmazione** e può essere eseguito su qualsiasi piattaforma, purché rispetti un protocollo di comunicazione standard (ad esempio HTTP, gRPC, ecc.) e il **'vocabolario' di messaggi** previsto dal servizio stesso.

Un servizio non implementa una interface come un POJO, ma definisce i messaggi che il servizio stesso può ricevere e inviare. Si veda anche *Qak visto da Gemini*.

1.5 Il problema della interazione

Un oggetto A che voglia usare (un oggetto B deve conoscere la interfaccia di B e interagisce con B tramite chiamate di metodi. In altre parole, A deve **trasferire il controllo** della CPU a B e attendere che B esegua il proprio codice e restituisca il controllo a A.

Se A e B sono scritti in linguaggi diversi, la interazione è possibile solo inviando/ricevendo informazione mediante l'uso di un protocollo di comunicazione e della rete.

Inoltre, A e B sono 'visti solo dal punto di vista esterno' e la loro interazione è, di base, **asincrona**.

1.6 Lo Sprint0

Sulla base delle premesse precedenti, cerchiamo di **impostare un metodo** per lo sviluppo dello Sprint0, il più delicato e strategicamente importante di tutti gli Sprint.

1.6.1 Sprint0: il Goal

Interagire con il committente per:

- definire in modo preciso e non ambiguo (formalizzare) i requisiti del sistema da sviluppare
- delineare un primo modello (eseguibile) del sistema
- impostare un primo insieme di TestPlan funzionali sulla base del modello dei requisiti
- definire il sottoinsieme di requisiti che fanno parte del goal dello Sprint1

1.6.2 Sprint0: le domande iniziali

1. Il sistema software da sviluppare è un sistema **distribuito** o un sistema centralizzato?
2. Se il sistema è distribuito, per quale numero minimo di ‘nodi computazionali’ diversi ($N > 1$) deve essere progettato il software?
3. Il software da sviluppare può essere espresso da un unico linguaggio di programmazione, o occorre usare più linguaggi diversi (sistema **eterogeneo**)?
4. Quali sono i macro-componenti software del sistema che devono essere sviluppati?
5. Quale è la ‘natura logica’ dei macro-componenti software del sistema? Sono oggetti, processi, servizi o altro?
6. Quali interazioni tra i macro-componenti sono esplicitamente indicate nel testo dei requisiti?
7. Come devono essere collocati i diversi componenti software sui vari nodi del sistema distribuito?
8. Come sono distribuite (guardando esclusivamente i requisiti, cioè escludendo ogni analisi del problema e/o progettazione) le **responsabilità funzionali evocate** dai requisiti tra i diversi componenti software del sistema?

Scopo del corso

Si ricorda che il corso (si veda CorsoISS2026) intende affrontare proprio il tema della costruzione di sistemi software distribuiti (eterogenei) capaci di superare l’idea che un sistema software sia formato solo da un insieme di oggetti (POJO) a favore dell’idea che i **macro-componenti** del sistema siano **(micro)servizi**.

1.6.3 Sprint0: parlare con la macchina

Tutte le risposte alle domande precedenti devono essere **formalizzate** (quando opportuno/necessario) in modo che siano comprensibili alla macchina.

A questo punto osserviamo che

- esiste un notevole **abstraction-gap** tra l’idea di componenti come (micro)servizi evocata dai requisiti e un ‘normale’ linguaggio di programmazione general purpose (Java, C#, Python, ecc.) che non è capace di esprimere in modo ‘nativo’ i vincoli di interazione tra i (macro)componenti del sistema da realizzare
- non esiste solo un problema relativo a ‘come si parla con la macchina’ ma anche un problema relativo ai **costi e tempi di sviluppo** (e poi di modifica e manutenzione) del prodotto software.

Più alto è l’abstraction gap, più sarà necessario scrivere codice ‘di contorno’ (**boilerplate code**) per ‘costruire’ ciò che non si può ‘esprimere’ in modo nativo.

1.6.4 Come ridurre il boilerplate code

Occorre ‘elevare’ il livello di espressività del linguaggio usato per ‘parlare con la macchina’, portandolo più vicino al livello di astrazione dei concetti espressi nei requisiti.

Fortunamente, questo obiettivo può essere raggiunto nel nostro processo di analisi dei requisiti grazie a un DSL definito dalla nostra ‘azienda’: il Linguaggio qak e la sua associata software factory.

Attraverso l’uso del QAk, possiamo esprimere la nostra analisi dei requisiti in modo **formale** (comprensibile ed anche eseguibile dalla macchina) definendo un **modello del sistema**.

E’ anche possibile definire in modo formale un primo **test plan** relativo alla interazione tra *IOPort* (che il committente vuole sia una web-gui) e il *cargoservice*.

i Modelli e Prototipi

Anche se il sistema deve essere distribuito su $N > 1$ nodi, nelle fasi iniziali di sviluppo è possibile organizzare ed eseguire il sistema in un **unico Context** che contiene vari QakActors che fungono da ‘mock’ services.

Una volta definita e testata la logica di interazione tra i componenti software, sarà poi possibile ‘estrarre’ un mock actor (**MockA**) dall’unico contesto e allocarlo in un nodo diverso.

Inoltre, **MockA** può essere realizzato con una **tecnologia completamente diversa** dai Qak: basta che sia mantenuta la logica di interazione a messaggi con il resto del sistema.

1.6.5 Sprint0: note specifiche

- cargorobot deve essere sviluppato in toto o il committente fornisce software?

Il committente fornisce un POJO (*RobotObj26*) e 3 servizi (*VirtualRobot26*, *VirtuRobotService26* e *RobotSmart26*). Occorre esaminare i vari componenti e capire quali di essi sia più adeguato per esprimere/realizzare il **cargorobot** menzionato nel testo.

- Led: deve essere quello sul PicoW
- IOPort: deve essere una web-gui

1.7 Qak visto da Gemini

Ho chiesto a Gemini quale costruito linguistico può essere la API di un microservizio nel caso questo sia espresso con il linguaggio qak.

La risposta di Gemini è stata:

La scelta del linguaggio **qak** (QActor meta-language) cambia completamente la prospettiva rispetto ai linguaggi general-purpose classici. Nel paradigma ad attori e orientato ai messaggi di qak, l’API di un microservizio non è più una collezione di metodi da chiamare, ma si trasforma in un **contratto di messaggi e transizioni di stato vincolato a un contesto di rete**.

In qak, l’API di un microservizio si esprime formalmente attraverso la combinazione di tre specifici costrutti linguistici nativi.

1.7.1 1. La Dichiarazione dei Messaggi (Il Contratto Funzionale)

Il primo vero livello di API in qak è la definizione dei messaggi in cima al file di modello. Questi costrutti definiscono *cosa* il microservizio è biologicamente in grado di ricevere e comprendere. L’API si esprime tramite tre forme primitive:

- **`Dispatch` (Asincrono One-Way):** Definisce un’API di tipo “fire-and-forget”. Il client invia un comando o un’informazione e non aspetta risposta (es. un comando di log o un aggiornamento di stato).
- **`Request` / `Reply` (Asincrono Two-Way):** È l’equivalente asincrono di una chiamata a funzione o di una REST API. Definisce un’API che richiede espressamente una risposta accoppiata.
- **`Event` (Publish-Subscribe):** Definisce un’API guidata dall’ambiente. Il microservizio non indirizza il messaggio a qualcuno, ma “emette” un fatto nel sistema; chi è interessato si sintonizza.

```
// Esempio di API funzionale espressa in qak
Dispatch assegnaIncarico : assegnaIncarico(DOCENTE, CORSO)
Request valutaCurriculum : valutaCurriculum(ID_DOCENTE)
Reply esitoValutazione : esitoValutazione(STATO)
```

1.7.2 2. Il *Context* (Il Confine del Microservizio e l'Endpoint di Rete)

Nei sistemi distribuiti tradizionali, l'API ha bisogno di un indirizzo IP e di una porta gestiti da file di configurazione esterni (es. YAML o Docker). In qak, l'estensione fisica dell'API è un **costrutto nativo del linguaggio**: il *Context*.

Il *Context* mappa un intero nodo logico (il microservizio) su un preciso indirizzo di rete. Tutti gli attori (*QActor*) definiti dentro quel contesto esporranno le proprie API di messaggistica attraverso quel singolo canale TCP/procedurale.

```
// L'indirizzo fisico in cui risponde l'API del microservizio
Context ctxGestionePersonale ip "127.0.0.1" port 8020
```

1.7.3 3. La transizione di Stato

Mentre nei linguaggi tradizionali l'API invoca un metodo che esegue il codice immediatamente, in qak l'API è regolata dallo **Stato dell'Attore**. Un microservizio può decidere di accettare un messaggio solo se si trova nello stato corretto, offrendo una gestione nativa della concorrenza e della sicurezza protetta da conflitti.

L'API "prende vita" nel blocco delle transizioni del *QActor*:

```
QActor enteagent context ctxGestionePersonale {
  State s0 initial { ... }
  Goto attendiRichieste

  State attendiRichieste { ... }
  // Il costrutto linguistico che attiva la risposta all'API
  Transition t0 whenRequest valutaCurriculum -> gestisciValutazione
}
```

1.7.4 In sintesi

Se in un'architettura classica orientata agli oggetti l'API è un'Interfaccia con firme di metodi, nel sistema distribuito in qak l'API del microservizio è:

Un insieme di **`Request/Dispatch`** atomici, esposti da un **`QActor`** localizzato dentro un **`Context`** di rete e regolati da una **`Transition`** di stato.

Questo approccio linguistico elimina alla radice i problemi di concorrenza (i famosi *Lock* di memoria condivisa), perché l'attore elabora un messaggio alla volta sfilandolo dalla sua coda interna in modo sequenziale, anche se la rete circostante è massivamente asincrona.